计算机应用
Journal of Computer Applications
ISSN 1001-9081,CN 51-1307/TP

《计算机应用》网络首发论文

题目：　基于历史数据和多目标优化的测试用例排序方法
作者：　李兴佳，杨秋辉，洪玫，潘春霞，刘瑞航
收稿日期：　2021-11-28
网络首发日期：　2022-05-18
引用格式：　李兴佳，杨秋辉，洪玫，潘春霞，刘瑞航．基于历史数据和多目标优化的测试用例排序方法[J/OL]．计算机应用．
https://kns.cnki.net/kcms/detail/51.1307.TP.20220517.1117.007.html

网络首发：在编辑部工作流程中，稿件从录用到出版要经历录用定稿、排版定稿、整期汇编定稿等阶段。录用定稿指内容已经确定，且通过同行评议、主编终审同意刊用的稿件。排版定稿指录用定稿按照期刊特定版式（包括网络呈现版式）排版后的稿件，可暂不确定出版年、卷、期和页码。整期汇编定稿指出版年、卷、期、页码均已确定的印刷或数字出版的整期汇编稿件。录用定稿网络首发稿件内容必须符合《出版管理条例》和《期刊出版管理规定》的有关规定；学术研究成果具有创新性、科学性和先进性，符合编辑部对刊文的录用要求，不存在学术不端行为及其他侵权行为；稿件内容应基本符合国家有关书刊编辑、出版的技术标准，正确使用和统一规范语言文字、符号、数字、外文字母、法定计量单位及地图标注等。为确保录用定稿网络首发的严肃性，录用定稿一经发布，不得修改论文题目、作者、机构名称和学术内容，只可基于编辑规范进行少量文字的修改。

出版确认：纸质期刊编辑部通过与《中国学术期刊（光盘版）》电子杂志社有限公司签约，在《中国学术期刊（网络版）》出版传播平台上创办与纸质期刊内容一致的网络版，以单篇或整期出版形式，在印刷出版之前刊发论文的录用定稿、排版定稿、整期汇编定稿。因为《中国学术期刊（网络版）》是国家新闻出版广电总局批准的网络连续型出版物（ISSN 2096-4188，CN 11-6037/Z），所以签约期刊的网络版上网络首发论文视为正式出版。

网络首发时间：2022-05-18 09:18:21
网络首发地址：https://kns.cnki.net/kcms/detail/51.1307.TP.20220517.1117.007.html

Journal of Computer Applications ISSN 1001-9081
计算机应用 CODEN JYIIDU http://www.joca.cn

doi:10.11772/j.issn.1001-9081. 2021112015

基于历史数据和多目标优化的测试用例排序方法

李兴佳，杨秋辉，洪玫*，潘春霞，刘瑞航

(四川大学 计算机学院，成都 610065)

(*通信作者电子邮箱 hongmei@scu.edu.cn)

摘　要: 针对如何提高测试用例序列的揭错效率和提升回归测试效益问题，提出了一种基于历史数据和多目标优化的测试用例排序方法。首先根据测试用例的文本主题相似性和代码覆盖相似性对测试用例集聚类，并根据历史执行信息对测试用例间的执行失败关系进行关联规则挖掘，为后续过程做准备；然后利用多目标优化算法对每个类簇内的测试用例排序，再生成最终排序序列，使相似的测试用例分隔开；最后利用测试用例间的关联规则，动态调整测试用例执行次序，使可能失败的测试用例优先执行，以进一步提高缺陷检测效率。与随机排序方法、基于聚类的排序方法、基于主题模型的排序方法、基于关联规则和多目标优化的排序方法对比，该方法的平均故障检测率（APFD）平均值分别提高了 12.60%，5.98%，3.01%和 2.95%，基于成本的平均故障检测率（APFDc）平均值分别提高了 17.18%，5.05%，5.08%和 8.21%。实验结果表明，该方法能有效提高回归测试效益。

关键词: 回归测试；测试用例聚类；关联规则挖掘；测试用例排序；多目标优化

中图分类号:TP311.5 **文献标志码: A**

Test case prioritization approach based on historical data and multi-objective optimization

LI Xingjia, YANG Qiuhui, HONG Mei*, PAN Chunxia, LIU Ruihang

(College of Computer Science，Sichuan University，Chengdu Sichuan 610065，China)

Abstract: Focused on the issue of how to improve the error detection efficiency of test case sequence and the benefit of regression testing, a test case prioritization approach based on historical data and multi-objective optimization was proposed. Firstly, test cases were clustered according to the text topic similarity and code coverage similarity of test cases, and association rules were mined for execution failure relationship between test cases according to historical execution information to prepare for the subsequent process. Then, the multi-objective optimization algorithm was used to sort the test cases in each cluster, and then the final sorting sequence was generated to separate the similar test cases. Finally, the association rules between test cases were used to dynamically adjust the execution order of test cases, so that the test cases that may fail were preferentially executed, so as to further improve the efficiency of defect detection. Compared with random search approach, the approach based on clustering, the approach based on topic model, the approach based on association rules and multi-objective optimization, the average values of Average Percentage of Faults-Detected (APFD) of this approach increased by 12.60%, 5.98%, 3.01% and 2.95%, and the average values of Average Percentage of Faults-Detected Cost-cognizant(APFDc) increased by 17.18%, 5.05%, 5.08% and 8.21%, respectively. Experimental results show that this approach can effectively improve the efficiency of regression testing.

Keywords: regression test; test case clustering; association rule mining; test case prioritization; multi-objective optimization

0　引言

回归测试是对修改后的软件进行重新测试，以确保未因修改代码而引入新的缺陷。由于软件不断演化，回归测试中的测试用例集会变得逐渐庞大与复杂，导致测试成本不断增大。在持续集成开发环境下，产品迭代速度不断加快，加剧了回归测试成本，如在 Google 中，开发人员平均每秒钟提交一次代码，每天需要执行超过 1.5 亿次测试[1]，测试成本较大。如何有效对测试用例集进行优化，减少测试用例执行成本，提升回归测试效益？这个问题吸引了众多国内外学者进行研究。

收稿日期: 2021-11-28;　**修回日期:** 2022-05-01;　**录用日期:** 2022-05-07。

作者简介: 李兴佳(1997—)，男（汉族），河北石家庄人，硕士研究生，主要研究方向：软件质量保证与测试；杨秋辉（1970—），女，山东青岛人，副教授，博士，主要研究方向：软件自动化测试、软件项目管理；洪玫(1963—)，女（通信作者），浙江舟山人，教授，硕士，CCF 高级会员（13579S），主要研究方向：软件分析与测试；潘春霞(1995—)，女，四川内江人，硕士，主要研究方向：软件质量保证与测试；刘瑞航(1998—)，男，黑龙江哈尔滨人，硕士研究生，主要研究方向：移动应用质量保证与测试。

回归测试用例优化技术可分为测试套件最小化技术、测试用例选择技术和测试用例排序技术[2]。测试用例排序是按照某种规则安排测试用例的执行次序，使具有高优先级的测试用例更早执行，以尽快检测出缺陷，提升回归测试效率。

本文在已有研究的基础上，提出了一种基于历史数据和多目标优化的测试用例排序方法：首先在历史测试数据分析阶段，将测试不同功能且具有不同代码覆盖范围的测试用例进行区分，同时挖掘测试用例间的关联规则；然后在多目标排序阶段，在聚类结果的基础上，使用多目标优化技术对测试用例集排序，以将相似的测试用例分隔开；最后在动态调整阶段，根据测试用例间的执行失败关联规则，动态调整测试用例执行顺序。本文提出的方法旨在提升测试的缺陷检测速率，使具有多样性和更有可能检测到缺陷的测试用例优先执行，从而使测试人员更快发现缺陷，提升回归测试效益。

1 相关工作

测试用例排序技术可分为使用静态信息的排序技术、使用动态信息的排序技术和使用混合信息的测试用例排序技术[3]。

Xiao 等[4]使用支持向量机预测测试用例检测到缺陷的可能性，并对测试用例聚类，最后进行簇内排序和簇间排序。Najafi等[5]对揭示更多缺陷和平均执行时间更少的测试用例赋予更高的优先级。Noor 等[6]使用多种距离度量方法计算测试用例相似性，并优先执行和历史执行失败的测试用例相似的测试用例。Busjaeger 等[7]根据测试用例的代码覆盖率、内容相似度、失败历史等信息，使用支持向量机对测试用例排序。De 等[8]根据测试用例在测试需求、测试依赖和测试步骤方面的相似性，去除相似性高的测试用例，减少了测试用例数量和测试执行时间，同时保证了测试用例的多样性。Carlson等[9]使用层次聚类算法对测试用例进行聚类，然后对每一类测试用例进行排序，生成最终的排序序列。Thomas 等[10]利用语义主题模型生成测试用例的主题分布，然后计算测试用例间的相似性，在排序过程中，将与已排序用例集具有更低相似性的测试用例优先排序。Pradhan 等[11]提出了一种利用关联规则和多目标搜索的黑盒测试用例排序方法，该方法首先挖掘测试用例的关联规则，然后使用 NSGA-II（Non-dominated Sorting Genetic Algorithm II）算法对测试用例排序，在按照该顺序执行过程中，根据关联规则，动态调整执行顺序。

目前的测试用例排序技术在判断测试用例相似性时，仅利用了单一的静态信息或动态信息；在利用多目标排序技术时，没有考虑测试用例冗余的问题，不利于提升排序序列的揭错速率。针对现有方法存在的不足，本文提出了一种基于历史数据和多目标优化的测试用例排序方法。

2 方法描述

为了提高检测速率，尽早发现错误，排序序列应满足以下目标：让测试不同功能且具有不同代码覆盖范围的测试用例尽早执行；在一定时间内尽可能执行更多的测试用例；使可能检测到缺陷的测试用例尽早执行。为了满足这些排序目标，设计的方法整体流程包括三个阶段，分别为历史测试数据分析、多目标排序和动态调整执行次序，整体流程如图 1 所示。

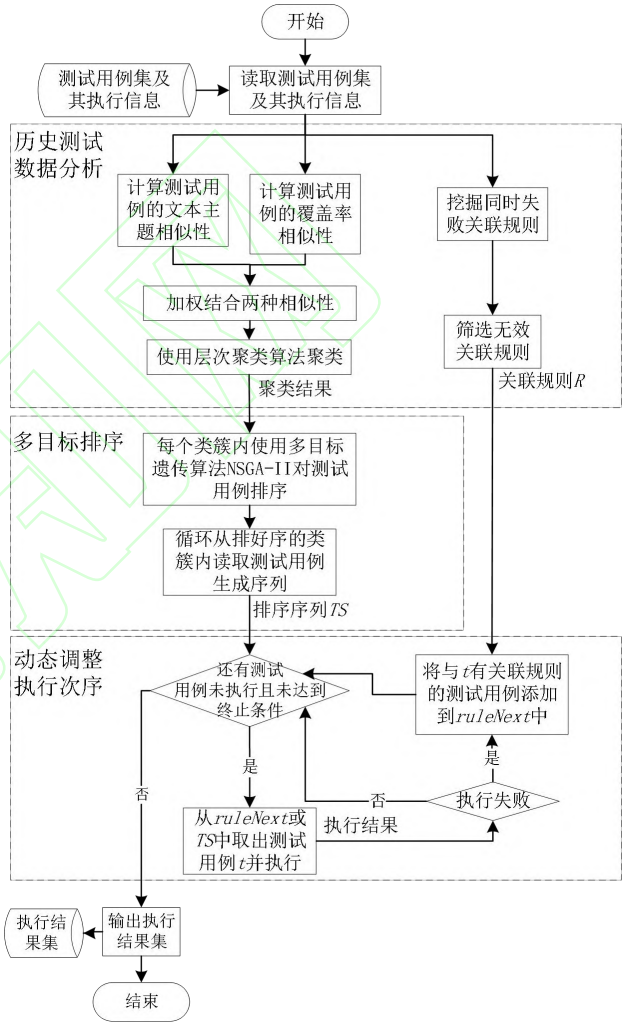

图 1 方法流程图

Fig. 1 Approach flow chart

2.1 基于文本主题和代码覆盖相似性对测试用例聚类

已有研究证明[9]，如果测试用例具有一些共同的属性(如代码覆盖范围)，那么同一组中的测试用例可能具有类似的故障检测能力。文献[9]通过实验对比了分别仅使用代码覆盖率、代码复杂性、有关实际故障的历史数据对测试用例进行排序和将上述指标分别与聚类算法结合后对测试用例排序两种方式的平均故障检测率（Average Percentage of Fault-Detection, APFD）值，结果表明结合聚类算法后，APFD

值有明显提升，其中代码覆盖率指标效果更好。因此将聚类算法应用于测试用例优先级排序技术，可以提升排序效果。

基于文本主题相似性的方法从测试用例静态文本的角度出发，考虑了测试用例在功能上的相似性，提升了排序效果[10]。但是仍旧存在这样的问题：对于如基于文本主题相似性的局限性示例中的被测方法和测试用例，测试用例 1 和测试用例 2 只有参数不同，从文本相似性角度分析，这两个测试用例属于同一类，但是这两个测试用例测试的代码块完全不同，不应归为同类测试用例，因此本文同时考虑了基于代码覆盖的相似性，从测试用例动态执行信息的角度出发，考虑测试用例语句级别代码覆盖范围的相同程度。为更全面地评价测试用例的相似性，在分别计算出基于文本主题和基于代码覆盖的相似性后，将这两种相似性进行加权结合。

基于文本主题相似性的局限性示例：

```
// 被测方法
public void func(int value){
    if(value>10){
        …
    }else{
        …
    }
}
// 测试用例 1
public void testFunc_1(){
    func(1)
}
// 测试用例 2
public void testFunc_2{
    func(100)
}
```

基于文本主题的测试用例相似性计算方法如下：将预处理后的测试用例文本信息集 $T_{pre}=\{tp_1, tp_2, \cdots, tp_n\}$（$tp_i$ 代表测试用例 t_i 的文本信息，n 为测试用例总数）使用隐含狄利克雷分布（Latent Dirichlet Allocation，LDA）模型进行主题建模，得到 $\boldsymbol{D}=\{\boldsymbol{d}_1, \boldsymbol{d}_2, \cdots, \boldsymbol{d}_n\}$，其中 $\boldsymbol{d}_i$ 表示测试用例 t_i 的主题分布。然后将主题分布 $\boldsymbol{d}_i$ 视为向量并计算向量之间的距离以度量测试用例的相似性，距离值越小，测试用例越相似。基于文本主题的测试用例相似性计算流程如图 2 所示，最终得到的相似矩阵是一个对称矩阵，其中 dis_{ij} 即为测试用例 t_i 与 t_j 之间的距离值。

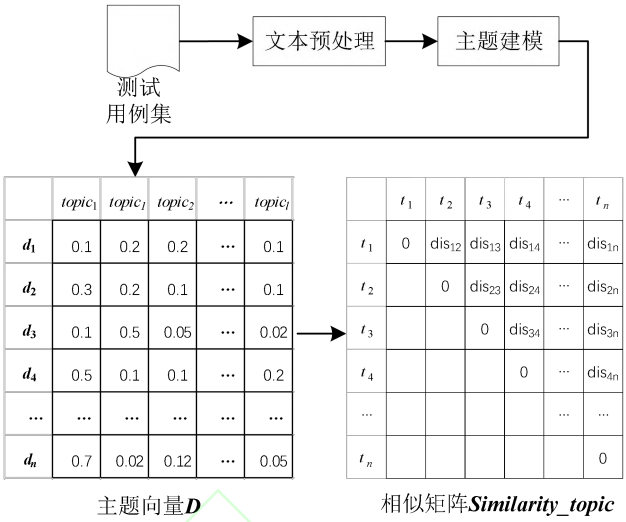

图 2 基于文本主题的测试用例相似性计算流程

Fig. 2 Test case similarity computing process based text topic

已有研究[12-13]证明，语句级别覆盖在测试用例排序技术中表现最好，因此，本文采用语句级别的覆盖来计算测试用例代码覆盖的相似性。首先需要统计测试用例执行后的语句覆盖情况 $TC=\{tc_1, tc_2, \cdots, tc_n\}$，其中 tc_i 表示测试用例 t_i 覆盖的语句集合。然后使用杰卡德距离[14]度量两个测试用例间的相似性，两个测试用例语句覆盖范围重合越大，杰卡德距离越小，也就代表两个测试用例越相似。最终得到一个与图 2 中结构相同的相似矩阵。

将这两种动静态相似性结合的表达式如式（1）所示：

$$\boldsymbol{Similarity_final} = \boldsymbol{Similarity_topic} * \alpha + \boldsymbol{Similarity_coverage} * (1-\alpha) \tag{1}$$

式中，$\boldsymbol{Similarity_topic}$ 是基于文本主题的相似矩阵，$\boldsymbol{Similarity_coverage}$ 是基于代码覆盖的相似矩阵，α 是权重系数，$\boldsymbol{Similarity_final}$ 是加权计算后的相似矩阵。

最后根据加权结合后的测试用例之间的相似性距离矩阵，使用层次聚类算法，将测试用例集中的测试用例划分为不同的类。聚类结果为 $C=\{c_1, c_2, \cdots, c_z\}$，其中 c_i 代表第 i 个类簇。

2.2 测试用例的关联规则挖掘

测试用例之间可能存在这样的关系：在历史执行中，当一个测试用例执行失败时，其余一个或多个测试用例往往同时也失败，那么可以认为这个测试用例和与它经常同时失败的测试用例之间存在关联规则，表示为 $t_i \rightarrow (t_{i1}, t_{i2}, \cdots, t_{ik})$，意思为当 t_i 执行失败时，$t_{i1}, t_{i2}, \cdots, t_{ik}$ 往往也失败，并且认为每个失败的测试用例都是由一个单独的错误引起的[11]。研究[5,11,15-16]表明利用这些隐含的关系，可以提升回归测试的有效性。

本文使用 Apriori 算法[17]挖掘历史执行信息中存在的关联规则，将每次执行结果中所有失败的测试用例看作一个事务，将事务中每一个失败过的测试用例看作一个项。该算法过程主要分为两步：1）通过迭代，检索出测试用例执行信息数据库中支持度不小于设定阈值的项集，这些项集被称作频繁项集；2）从频繁项集中获取置信度不小于设定阈值的关联规则。其中支持度公式如式（2）所示：

$$support(A \to B) = \frac{number(AB)}{number(AllRecord)} \tag{2}$$

式中 $number(AB)$ 表示的是同时出现测试用例 A 和 B 的事务的总数，$number(AllRecord)$ 表示的是总事务数，$support(A \to B)$ 表示关联规则 $(A \to B)$ 的支持度，即 A 和 B 同时发生的概率。

置信度公式如式（3）所示：

$$confidence(A \to B) = \frac{number(AB)}{number(A)} \tag{3}$$

式中 $number(AB)$ 表示的是同时出现测试用例 A 和 B 的事务的总数，$number(A)$ 表示的是出现测试用例 A 的事务总数，$confidence(A \to B)$ 表示关联规则 $(A \to B)$ 的置信度，即在 A 发生的情况下，B 发生的条件概率。

在挖掘到的关联规则中可能存在这样的情况：某些存在关联规则的测试用例揭示的是相同的缺陷，这种情况并不能有效提升揭错效率，因此需要进行筛选。筛选过程的核心思想是：若两个测试用例揭示相同的缺陷次数超过共同失败次数的一半，那么在 A 揭示这个缺陷后，就没有必要立即执行关联的测试用例 B 了，因为 B 在很大概率下也会揭示相同的缺陷，因此应将其视为无效。在挖掘关联规则后，对这种类型的关联规则进行筛选，本文仅保留有效的关联规则，用于调整测试用例执行次序。最终得到关联规则集合如 $R=\{t_1 \to (t_2, t_3)，t_{21} \to (t_{12})，\cdots，t_i \to (t_{i1}，t_{i2}，\cdots，t_{ik})\}$。

2.3 多目标排序和动态调整执行次序

Epitropakis 等[18]通过对比使用多目标优化算法和其他方法的排序效果，证明了使用多目标优化算法解决测试用例排序问题的有效性。多目标优化问题是求出使各个目标尽可能最优的解，这种解通常是一组 pareto 最优解。多目标优化[19]问题的表达式如式（4）所示：

$$\begin{aligned} &\max\ F(x) = (f_1(x), f_2(x), \ldots, f_e(x)) \\ &s.t. x \in \Omega \end{aligned} \tag{4}$$

式中，$x = (x_1, x_2, \ldots, x_h) \in \Omega$ 是 h 维决策变量，Ω 是决策空间，表示每一维决策向量的上下界，e 是子目标数，$f_1(x)$ 至 $f_e(x)$ 是各个子目标优化函数。

本文采用带精英策略的非支配排序遗传算法 NSGA-II[20]在每个类簇内进行多目标排序。该算法具有运行速度快、解集收敛性好的优点，目前在多目标测试用例优先级排序领域中应用较为广泛。首先在每个类簇内进行多目标排序，得到 $TS_c_i=\{t_1'，t_2'，\cdots，t_h'\}$，$TS_c_i$ 表示类簇 c_i 的排序序列；然后依次从每个排序序列中抽取测试用例，生成最终的排序序列 TS。

本文中使用的优化子目标分别为代码覆盖率最大化、历史执行失败率最大化以及执行时间最小化，在迭代过程中，越满足这三个目标的个体，越容易被保留下来。

代码覆盖率最大化的函数表达式如式（5）所示：

$$\max\ f1 = \frac{\sum_{i=1}^{h} SC_{ti} \times \frac{h-i+1}{h}}{ssc} \tag{5}$$

式中，SC_{ti} 表示测试用例 t_i 的语句覆盖数量，h 代表排序测试用例集的数量，i 代表测试用例在排序序列中位置，ssc 为测试用例集中所有测试用例语句覆盖数量的和。

历史执行失败率最大化的函数表达式如式（6）所示：

$$\max\ f2 = \frac{\sum_{i=1}^{h} FR_{ti} \times \frac{h-i+1}{h}}{sfr} \tag{6}$$

式中，FR_{ti} 表示测试用例 t_i 的历史执行失败率，sfr 为测试用例集中所有测试用例 FR 值的和，其余参数含义同式（5）中参数一致。

执行时间最小化的函数表达式如式（7）所示，

$$\max\ f3 = \frac{\sum_{i=1}^{h} TIME_{ti} \times \frac{i}{h}}{st} \tag{7}$$

式中 $TIME_{ti}$ 表示测试用例 t_i 的执行时间，st 表示执行时间总和，其余参数含义同式（5）中参数一致。

生成最终排序序列的过程为：循环遍历每个类簇内的排序序列 TS_c_i，对于第 j 轮遍历，若 TS_c_i 的长度大于等于 j，则抽取第 j 个测试用例添加到 TS 的末尾；否则，跳过 TS_c_i，直至遍历完所有的测试用例，得到最终的排序序列 $TS=\{t_1，t_2，\cdots，t_n\}$。最终的排序序列靠前部分既保留了多样性，又保留了每一类中最有可能发现缺陷的测试用例，增大了快速发现不同缺陷的机会。

为了提升执行序列的缺陷检测速度，当测试用例执行时，根据测试用例间的执行失败关联规则去动态调整测试用例执行次序，让更可能检测到缺陷的测试用例优先执行。

使用关联规则动态调整测试用例排序次序的主要流程为：在按排序序列 TS 执行过程中，若当前测试用例 t_i 执行失败，若有关联规则 $t_i \to (t_{i1}，t_{i2}，\cdots，t_{ik})$,则立即执行 $t_{i1}，t_{i2}，\cdots，t_{ik}$ 中尚未执行的测试用例，然后再依次执行 TS 序列中剩余测试用例，直至达到终止条件或测试用例全部执行完毕。以哈

希表的形式存储测试用例和关联规则，其中每个测试用例仅被执行一次，时间复杂度是 O(n)，n 为测试用例总数；获取 k 个相关联的测试用例的时间复杂度为 O(k)，因此总的时间复杂度为 O(nk)；需要存储的变量有排序序列、关联规则和根据关联规则调整的测试用例，因此空间复杂度是 O(n)。动态调整过程如算法 1 所示：

算法 1 动态调整执行次序

输入 排序序列 $TS=\{t_1, t_2, \cdots, t_n\}$，关联规则 $R=\{t_i \rightarrow (t_{i1}, t_{i2}, \cdots, t_{ik})\}$，测试用例总数 n

输出 执行信息 *Record*

```
current = 1   // 记录当前执行的测试用例下标
ruleNext = Empty Queue
  // 失败时获取相关联的测试用例，添加到 ruleNext 中
while(current<=n AND not_meet_end_condition）do
    // 若测试用例未执行完和未达到终止条件
    if(ruleNext_is_not_empty) then
        // 若 ruleNext 不为空，从 ruleNext 取出测试用例
        t ← get a test case from ruleNext
    else
        // 否则，从 TS 中获取下标为 current 的测试用例
        t ← get TS[current]
        current ← current+1
    if(t_has_not_been_executed) then
        Result = execute(t)
        if(Result_is_fail) then
          // 若 t 的执行结果是失败
            Trules ← Get related test case
                //获取与 t 相关联的测试用例
            ruleNext ← Add(Trules)
                //添加到 ruleNext 中
        Record(Result)
          //记录 t 的执行信息，并将 t 标记为已执行过
return Record   //返回执行信息
```

3 实验验证

3.1 实验对象

SIR（Software-artifact Infrastructure Repository）库[21]是一个广泛地应用于回归测试用例集优化研究的开源数据库。本文从 SIR 库和 GitHub 共获取 5 个 Java 开源项目作为实验对象。表 1 介绍了实验的项目信息，其中“新版本缺陷数”代表每个项目最新版本所包含的缺陷数。“历史缺陷数”代表项目在历史版本中所揭示缺陷的总数，“历史版本数”也为“历史执行记录数”，是将测试用例在每个历史版本上的执行结果作为一次执行记录。获取自 SIR 库的 ant、derby 和 jboss 项目历史版本数相对较少，因此将缺陷数较多的版本划分为多个版本，以增加历史版本数。

表 1 实验项目信息

Tab. 1 Information of experiment projects

项目名称	测试用例数	新版本缺陷数	历史缺陷数	历史版本数
ant	105	9	13	8
derby	120	19	16	9
jboss	142	10	45	15
commons-lang	158	8	303	100
joda-time	126	9	378	100

3.2 评估指标

评价指标选择平均故障检测率（Average Percentage of Fault-Detection，APFD）和基于成本的平均故障检测率（Average Percentage of Fault-Detection Cost-cognizant，APFDc）。APFD 是指测试套件在执行过程中检测到缺陷的百分比。该评价指标被广泛地应用于评估测试用例排序技术。其计算方法如式（8）所示：

$$APFD = 1 - \frac{TF_1 + TF_2 + \ldots + TF_m}{mn} + \frac{1}{2n} \tag{8}$$

式中，m 为缺陷的总个数，TF_i 表示首次揭示第 i 个缺陷的测试用例在排序序列中的位置，n 代表测试用例数量。检测到不同缺陷的测试用例越排在前面，APFD 值越大，证明排序序列的揭错速度越快，对应的排序方法效果也越好。

APFDc 增加了对测试用例成本和缺陷严重程度的度量。文献[18,22]对传统的 APFDc 进行简化，仅考虑测试用例执行时间，而将缺陷严重程度都视为 1。本文采用简化后的 APFDc 指标，其计算公式如式（9）所示：

$$APFDc = \frac{\sum_{i=1}^{m}\left(\sum_{j=TF_i}^{n} TIME_j - \frac{1}{2}TIME_{TF_i}\right)}{\sum_{j=1}^{n} TIME_j \times m} \tag{9}$$

式中 m、n、TF_i 表达的含义与式（8）中一致，$TIME_j$ 代表执行第 j 个测试用例所花费时间，同 APFD 值一样，APFDc 值越大，对应排序方法效果越好。

3.3 实验设计

执行本文方法。首先使用层次聚类算法对测试用例集聚类，经实验验证，α 值为 0.7 时，效果最好；使用 Aprior 算法挖掘关联规则，设置最小置信度 *minconf* 为 0.8，由于不同项目具有的历史执行记录数不同，对应的项集频繁程度不同，设置最小支持度 *minsup* 的取值范围为 0.1-0.5。然后使用 NSGA-II 算法在每个类簇内对测试用例排序，再生成最终的排序序列。最后动态调整测试用例执行次序，直至所有测试用例执行完毕或达到终止条件。统计执行结果的 APFD 值和 APFDc 值。

为证明方法的有效性，在 5 个 Java 项目上，将本文方法与下述几种方法的排序结果进行对比。

随机排序方法：将测试用例集以随机顺序执行，并统计执行结果的 APFD 值和 APFDc 值。由于该方法存在随机性因素，因此记录 30 次执行结果进行比较。

文献[9]方法：执行 Carlson 等提出的基于聚类的排序方法[9]，并统计执行结果的 APFD 和 APFDc 值，由于该方法不存在随机性因素，该方法仅需执行一次。

文献[10]方法：执行 Thomas 等提出的基于主题模型的排序方法[10]，并统计执行结果的 APFD 和 APFDc 值。由于使用 LDA 模型推导测试用例主题分布时，使用了随机数种子，因此记录该方法 30 次执行结果进行比较。

文献[11]方法：执行 Pradhan 等提出的基于关联规则和多目标优化的排序方法[11]，并统计执行结果的 APFD 和 APFDc 值。由于多目标遗传算法具有随机性，因此记录该方法的 30 次执行结果进行比较。

3.4 实验结果与分析

本文方法综合利用了测试用例的静态信息和动态信息。基于测试用例静态信息的处理主要包括计算测试用例文本主题和代码覆盖相似性、层次聚类与多目标排序，在测试用例本身未改变之前，上述过程的结果可以重复使用。基于动态信息的处理主要包括关联规则挖掘与动态调整执行次序两个过程，其中动态调整执行次序到的时间复杂度为 O(nk)。

3.4.1 APFD 结果

表 2 上半部分展示了在 5 个项目上分别使用 5 种排序方法所得到的平均 APFD 值：

表 2 不同项目使用不同排序方法的平均 APFD 值和 APFDc 值对比

Tab. 2 Comparison of the average APFD values and APFDc values of different projects using different prioritization methods

评估指标	项目名称	随机方法	文献[9]方法	文献[10]方法	文献[11]方法	本文方法
APFD	ant	0.6785	0.7290	0.8115	0.7632	**0.8422**
	derby	0.8321	0.9081	0.9029	0.9094	**0.9123**
	jboss	0.8892	0.9013	0.9242	0.9180	**0.9309**
	commons-lang	0.7218	0.7758	0.7832	**0.8125**	0.8115
	joda-time	0.7497	0.7987	0.8098	0.8307	**0.8620**
	平均值	0.7743	0.8226	0.8463	0.8468	**0.8718**
APFDc	ant	0.7096	0.7534	0.8090	0.7471	**0.8259**
	derby	0.8311	0.9269	**0.9329**	0.8172	0.9215
	jboss	0.8680	0.9213	0.9045	0.8950	**0.9409**
	commons-lang	0.7472	0.9350	0.8529	0.8941	**0.9511**
	joda-time	0.7489	0.8192	0.8550	0.8749	**0.9362**
	平均值	0.7810	0.8712	0.8709	0.8457	**0.9151**

从表中可以看出，在大部分项目中，本文方法得到的 APFD 值更高（黑体部分）。尤其是在 ant 项目中，本文方法相较于随机方法提升率为 24.13%，提升明显，相较于文献[9]、文献[10]和文献[11]的方法，提升率分别为 15.53%、3.78%和 10.35%。在 derby 项目中，本文方法相较于除随机方法外的其他方法优势不明显，是因为该项目中部分测试用例能单独检测出较多的缺陷，这类测试用例被排在靠前位置时，总能导致排序结果的 APFD 值较高，在 jboss 项目中，相较于文献[10]和文献[11]方法提升不明显原因也是如此。在 commons-lang 项目中，本文方法相较于文献[11]方法表现更差一些，通过分析项目发现：该项目的测试用例间的相似性较差，而本文使用了相似性进行聚类，文献[11]方法未考虑相似性，因此本文方法排序效果更差。相比其他几种方法，本文方法在所有项目上的平均值更高，分别提高了 12.60%、5.98%、3.01%和 2.95%。

3.4.2 APFDc 值结果

表 2 下半部分展示了 5 个项目分别使用 5 种排序方法所得到的平均 APFDc 值。

从表中可以看出，在大部分项目中，本文方法得到的 APFDc 值更高。与随机方法相比，本文方法得到的 APFDc 值在各个项目中都提升明显，在 commons-lang 项目中的提升率最高，达到 27.29%。在 derby 项目中，本文方法相比于文献[10]方法表现得更差一些，通过分析发现：该项目的测试用例集中存在执行时间较长（约为 98 秒）但在历史执行结果中多次发现缺陷的测试用例，这两个性质在多目标排序阶段互相制约，导致本文方法在该项目上的效果比文献[10]方法的差。相较于文献[9]、文献[10]和文献[11]的方法，本文方法的 APFDc 值在所有项目上的最高提升率分别为 14.28%、11.51%和 12.76%。相比其他几种方法，本文方法在所有项目上的平均值更高，分别提高了 17.17%、5.04%、5.08%和 8.21%。综合上述情况，证明了本文方法的有效性。

4 结语

针对如何提升回归测试效益问题，提出了一种综合利用历史数据和执行中信息的测试用例排序方法。首先利用测试用例文本信息和历史执行信息，对测试用例进行聚类和关联规则挖掘，然后利用多目标优化算法对测试用例排序和调整，最后在执行过程中，根据测试用例的执行信息对序列进行进一步优化。通过比较本文方法与其他 4 个方法可知，本文方法的 APFD 和 APFDc 值具有一定的提高。

本文所提方法还存在一些不足，未来可深入进行研究，包括但不局限于：在多目标排序阶段，合理增加优化子目标，如在实际的工业项目中，还会考虑需求覆盖等目标，以进一步研究加入这些目标后的排序效果；本文实验所用数据集的历史记录不够庞大，对关联规则的挖掘具有一定的影响，未来可考虑在具有庞大历史执行记录的实际工业项目中验证本文的方法。

参考文献

[1] LEONG C, SINGH A, PAPADAKIS M, et al. Assessing transition-based test selection algorithms at Google[C]//ICSE-SEIP: Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice. Piscataway：IEEE,2019: 101-110.

[2] YOO S, HARMAN M. Regression testing minimization, selection and prioritization: a survey[J]. Software Testing, Verification and Reliability, 2012, 22(2): 67-120.

[3] LUO Q, MORAN K, ZHANG L, et al. How do static and dynamic test case prioritization techniques perform on modern software systems? an extensive study on GitHub projects[J]. IEEE Transactions on Software Engineering, 2018, 45(11): 1054-1080.

[4] XIAO L, MIAO H, ZHUANG W W, et al. Applying assemble clustering algorithm and fault prediction to test case prioritization[C]// SATE: Proceedings of the 2016 International Conference on Software Analysis, Testing and Evolution. Piscataway: IEEE, 2016:108-116.

[5] NAJAFI A, SHANG W, RIGBY P C. Improving test effectiveness using test executions history: an industrial experience report[C]// ICSE-SEIP: Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice. Piscataway: IEEE, 2019: 213-222.

[6] NOOR T B, HEMMATI H. A similarity-based approach for test case prioritization using historical failure data[C]//ISSRE: Proceedings of the 2015 IEEE 26th International Symposium on Software Reliability Engineering. Piscataway: IEEE, 2015:58-68.

[7] BUSJAEGER B, XIE T. Learning for test prioritization: an industrial case study[C] //Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering. New York: ACM, 2016: 975-980.

[8] DE OLIVEIRA NETO F G, AHMAD A, LEIFLER O, et al. Improving continuous integration with similarity-based test case selection[C]//AST: Proceedings of the 2018 IEEE/ACM 13th International Workshop on Automation of Software Test. New York: ACM. 2018: 39-45.

[9] CARLSON R, DO H, DENTON A. A clustering approach to improving test case prioritization: an industrial case study[C]//ICSM: Proceedings of the 2011 27th IEEE International Conference on Software Maintenance. Piscataway: IEEE, 2011: 382-391.

[10] THOMAS S W, HEMMATI H, HASSAN A E, et al. Static test case prioritization using topic models[J]. Empir Software Eng, 2014, 19: 182-212.

[11] PRADHAN D, SHUAI W, ALI S, et al. REMAP: Using rule mining and multi-objective search for dynamic test case prioritization[C]// Proceedings of the 2018 IEEE 11th International Conference on Software Testing, Verification and Validation. Piscataway: IEEE, 2018: 46-57.

[12] HAO D, ZHANG L, ZHANG L U, et al. A unified test case prioritization approach[J]. ACM transactions on software engineering and methodology, 2015, 24(2): 10.1-10.31.

[13] MEI H, HAO D, ZHANG L, et al. A static approach to prioritizing junit test cases[J]. IEEE Transactions on Software Engineering, 2012, 38(6): 1258-1275.

[14] JIANG B, ZHANG Z, CHAN W, et al. Adaptive random test case prioritization[C]// Proceedings of the 2009 IEEE/ACM International Conference on Automated Software Engineering. Piscataway: IEEE, 2009: 233-244.

[15] ANDERSON J, SALEM S, DO H. Improving the effectiveness of test suite through mining historical data[C]// Proceedings of the 11th Working Conference on Mining Software Repositories. New York: ACM, 2014: 142–151.

[16] ZHU Y, SHIHAB E, RIGBY P C. Test re-prioritization in continuous testing environments[C]// Proceedings of the 2018 IEEE International Conference on Software Maintenance and Evolution. Piscataway: IEEE, 2018: 69-79.

[17] 李晓熠,周鋆. 基于频繁项挖掘的贝叶斯网络结构学习算法 BNSL-FIM[J]. 计算机应用, 2021, 41(12): 3475-3479.(LI X Y, ZHOU Y. BNSL-FIM: Bayesian network structure learning algorithm based on frequent item mining[J]. Journal of Computer Applications, 2021, 41(12): 3475-3479.)

[18] EPITROPAKIS M G, YOO S, HARMAN M, et al. Empirical evaluation of pareto efficient multi-objective regression test case prioritization[C]// Proceedings of the 2015 International Symposium on Software Testing and Analysis. New York: ACM, 2015: 234-245.

[19] 李二超,毛玉燕. 基于空间收缩技术的约束多目标进化算法[J]. 计算机应用, 2021, 41(12): 3419-3425.(LI E C, MAO Y Y. Constrained multi-objective evolutionary algorithm based on space shrinking technique[J]. Journal of Computer Applications, 2021, 41(12): 3419-3425.)

[20] DEB K, PRATAP A, AGARWAL S, et al. A fast and elitist multi-objective genetic algorithm: NSGA-II[J]. IEEE Transactions on Evolutionary Computation, 2002, 6(2): 182-197.

[21] DO H, ELBAUM S, ROTHERMEL G. Supporting controlled experimentation with testing techniques: an infrastructure and its potential impact[J]. Empirical Software Engineering, 2005, 10(4): 405-435.

[22] LIANG J, ELBAUM S, ROTHERMEL G. Redefining prioritization: continuous prioritization for continuous integration[C]// Proceedings of the 2018 IEEE/ACM 40th International Conference on Software Engineering. Piscataway: IEEE, 2018: 688-698.

LI Xingjia, born in 1997, M. S. candidate. His research interests include software quality assurance and testing.

YANG Qiuhui，born in 1970，Ph.D.，associate professor. Her research interests include software automation testing，software project management

HONG Mei, born in 1963, M.S., professor. Her research interests include software analysis and testing.

PAN Chunxia，born in 1995，M. S. Her research interests include software quality assurance and testing.

LIU Ruihang, born in 1998, M. S. candidate. His research interests include mobile application quality assurance and testing.